

Map-Reduce Concurrency

NOTE

Aggregation Pipeline as an Alternative to Map-Reduce

Starting in MongoDB 5.0, map-reduce is deprecated:

- Instead of map-reduce, you should use an aggregation pipeline. Aggregation pipelines provide better performance and usability than map-reduce.
- You can rewrite map-reduce operations using aggregation pipeline stages, such as `$group`, `$merge`, and others.
- For map-reduce operations that require custom functionality, you can use the `$accumulator` and `$function` aggregation operators. You can use those operators to define custom aggregation expressions in JavaScript.

For examples of aggregation pipeline alternatives to map-reduce, see:

- [Map-Reduce to Aggregation Pipeline](#)
- [Map-Reduce Examples](#)

The map-reduce operation is composed of many tasks, including reads from the input collection, executions of the `map` function, executions of the `reduce` function, writes to a temporary collection during processing, and writes to the output collection.

During the operation, map-reduce takes the following locks:

- The read phase takes a read lock. It yields every 100 documents.
- The insert into the temporary collection takes a write lock for a single write.
- If the output collection does not exist, the creation of the output collection takes a write lock.
- If the output collection exists, then the output actions (i.e. `merge`, `replace`, `reduce`) take a write lock. This write lock is *global*, and blocks all operations on the `mongod` instance.